

Лабораторна робота №7.

Обробка даних із сенсорів у багатозадачній системі

Мета роботи. Навчитися організовувати взаємодію між задачами у системі реального часу, що одночасно працюють із кількома сенсорами.

Хід виконання роботи.

1. Створіть проєкт FreeRTOS у середовищі розробки VS Code.
2. Завантажте в каталог вашого проєкту бібліотеки для роботи з OLED дисплеєм та сенсорами перейшовши за посиланням:

<https://github.com/mpavlyk/rtos/blob/main/lib/lib.zip>

3. Відкрийте файл **CMakeLists.txt**, який знаходиться у каталозі вашого проєкту і відредагуйте його наступним чином:

- знайдіть рядок із інструкцією `pico_sdk_init()`. Після цієї інструкції вставте:

1	# Add Subdirectory
2	add_subdirectory(/lib/Config)
3	add_subdirectory(/lib/OLED)
4	add_subdirectory(/lib/Fonts)
5	add_subdirectory(/lib/GUI)
6	add_subdirectory(/lib/SHTC3)
7	add_subdirectory(/lib/SGP40)
8	add_subdirectory(/lib/QMI8658)
9	
10	# Include Directories
11	include_directories(/lib/Config)
12	include_directories(/lib/OLED)
13	include_directories(/lib/Fonts)
14	include_directories(/lib/GUI)
15	include_directories(/lib/SHTC3)
16	include_directories(/lib/SGP40)
17	include_directories(/lib/QMI8658)

- Знайдіть інструкцію - `target_link_libraries(lab01 pico_stdlib)` і додайте бібліотеки - `hardware_spi`, `hardware_i2c`, `hardware_pwm`, `hardware_adc`, `Config`, `GUI`, `Fonts`, `OLED`.

1	<code>target_link_libraries(lab</code>
---	--

2	pico_stdlib
3	FreeRTOS-Kernel
4	FreeRTOS-Kernel-Heap4
5	hardware_spi
6	hardware_i2c
7	hardware_pwm
8	hardware_adc
9	Config
10	GUI
11	Fonts
12	OLED
13	SGP40
14	SHTC3
15	QMI8658)

4. Проаналізуйте, скомпілюйте та завантажте наведений код у Pico. В даному коді наведений приклад читання даних із сенсора температури та вологості і виведення на дисплей.

1	#include <stdio.h>
2	#include "pico/stdlib.h"
3	
4	#include "OLED_1in5.h"
5	#include "GUI_Paint.h"
6	#include "DEV_Config.h"
7	#include "SHTC3.h"
8	
9	int main()
10	{
11	float temp, hum;
12	int16_t id;
13	bool crc_res;
14	uint8_t *BlackImage;
15	uint16_t Imagesize = ((OLED_1in5_WIDTH % 2 == 0) ? (OLED_1in5_WIDTH /
16	2) : (OLED_1in5_WIDTH / 2 + 1)) * OLED_1in5_HEIGHT;
17	
18	if (DEV_Module_Init() != 0)
19	{
20	return -1;
21	}
22	//Initialize the OLED
23	OLED_1in5_Init();
24	OLED_1in5_Clear(0);
25	// Initialize the temperature and humidity sensor

```

26  SHTC3_Init();
27  if ((BlackImage = (uint8_t *)malloc(Imagesize)) == NULL)
28  {
29      while (1)
30      {
31          printf("Failed to apply for black memory...\r\n");
32      }
33  }
34  printf("Paint_NewImage\r\n");
35  Paint_NewImage(BlackImage, OLED_1in5_WIDTH, OLED_1in5_HEIGHT, 0,
36  WHITE);
37  Paint_SetScale(16);
38  Paint_SelectImage(BlackImage);
39
40  while (1)
41  {
42      char str[20];
43      Paint_Clear(BLACK);
44      crc_res = SHTC3_Measurement(&temp, &hum);
45      id = SHTC3_Read_Id();
46      // Read and validate the data
47      if (id == -1 || crc_res == false)
48      {
49          Paint_DrawString_EN(5, 40, "CRC8 fail", &Font12, WHITE, BLACK);
50          OLED_1in5_Display(BlackImage);
51          printf("Error: Measurement CRC8 check failure \r\n");
52          break;
53      }
54      Paint_DrawString_EN(5, 10, "T&H TEST", &Font12, WHITE, BLACK);
55      Paint_DrawString_EN(5, 25, "IC:SHTC3", &Font12, WHITE, BLACK);
56
57      sprintf(str, "ID : 0x%x", id);
58      Paint_DrawString_EN(5, 40, str, &Font12, WHITE, BLACK);
59
60      sprintf(str, "TEMP : %.2f", temp);
61      Paint_DrawString_EN(5, 55, str, &Font12, WHITE, BLACK);
62      sprintf(str, "RH : %.2f", hum);
63      Paint_DrawString_EN(5, 70, str, &Font12, WHITE, BLACK);
64
65      printf("T= %.2f , RH = %.2f \r\n", temp, hum);
66      OLED_1in5_Display(BlackImage);
67      DEV_Delay_ms(1000);
68  }
69  }

```

5. Проаналізуйте, скомпілюйте та завантажте наведений код у Pico. В даному коді наведений приклад читання даних із сенсора якості повітря і виведення на дисплей.

```
1 #include <stdio.h>
2 #include "pico/stdlib.h"
3 #include "OLED_1in5.h"
4 #include "GUI_Paint.h"
5 #include "DEV_Config.h"
6 #include "SGP40.h"
7
8 int main()
9 {
10     uint8_t *BlackImage;
11     uint16_t Imagesize = ((OLED_1in5_WIDTH % 2 == 0) ? (OLED_1in5_WIDTH /
12 2) : (OLED_1in5_WIDTH / 2 + 1)) * OLED_1in5_HEIGHT;
13
14     if (DEV_Module_Init() != 0)
15     {
16         return -1;
17     }
18     //Initialize the OLED
19     OLED_1in5_Init();
20     OLED_1in5_Clear(0);
21     // Initialize the VOL sensor
22     SGP40_init();
23     if ((BlackImage = (uint8_t *)malloc(Imagesize)) == NULL)
24     {
25         while (1)
26         {
27             printf("Failed to apply for black memory...\r\n");
28         }
29     }
30     printf("Paint_NewImage\r\n");
31     Paint_NewImage(BlackImage, OLED_1in5_WIDTH, OLED_1in5_HEIGHT, 0,
32 WHITE);
33     Paint_SetScale(16);
34     Paint_SelectImage(BlackImage);
35
36     while (1)
37     {
38         uint32_t voc;
39         char str[20];
40         Paint_Clear(BLACK);
41         // read raw and calculate the VOL
```

```

42     voc = SGP40_MeasureVOC(25, 50);
43     printf("\r\n /-----/\r\n");
44     printf("\r\n SGP40_MeasureVOC : %d\r\n",voc);
45     Paint_DrawString_EN(5, 10, "VOC TEST", &Font12, WHITE, BLACK);
46     Paint_DrawString_EN(5, 25, "IC:SGP40", &Font12, WHITE, BLACK);
47
48     sprintf(str, "VOC : %d ", voc);
49     Paint_DrawString_EN(5, 40, str, &Font12, WHITE, BLACK);
50     OLED_1in5_Display(BlackImage);
51 }
52 }

```

6. Проаналізуйте, скомпілюйте та завантажте наведений код у Pico. В даному коді наведений приклад читання даних із сенсора акселерометра та гіроскопа і виведення на дисплей.

```

1  #include <stdio.h>
2  #include "pico/stdlib.h"
3
4  #include "OLED_1in5.h"
5  #include "GUI_Paint.h"
6  #include "DEV_Config.h"
7  #include "QMI8658.h"
8
9  int main()
10 {
11     float acc[3], gyro[3];
12     unsigned int tim_count = 0;
13     uint8_t *BlackImage;
14     uint16_t Imagesize = ((OLED_1in5_WIDTH % 2 == 0) ? (OLED_1in5_WIDTH /
15 2) : (OLED_1in5_WIDTH / 2 + 1)) * OLED_1in5_HEIGHT;
16
17     if (DEV_Module_Init() != 0)
18     {
19         return -1;
20     }
21     //Initialize the OLED
22     OLED_1in5_Init();
23     OLED_1in5_Clear(0);
24
25     if ((BlackImage = (uint8_t *)malloc(Imagesize)) == NULL)
26     {
27         while (1)
28         {

```

```

29     printf("Failed to apply for black memory...\r\n");
30 }
31 }
32 printf("Paint_NewImage\r\n");
33 Paint_NewImage(BlackImage, OLED_1in5_WIDTH, OLED_1in5_HEIGHT, 0,
34 WHITE);
35 Paint_SetScale(16);
36
37 printf("Drawing\r\n");
38 // 1.Select Image
39 Paint_SelectImage(BlackImage);
40 Paint_Clear(BLACK);
41 //Initialize the 6DOF sensor
42 QMI8658_init();
43 while (true)
44 {
45     char str[20];
46     QMI8658_read_xyz(acc, gyro, &tim_count);
47     printf("acc_x = %4.3fmg , acc_y = %4.3fmg , acc_z = %4.3fmg\r\n", acc[0],
48 acc[1], acc[2]);
49     printf("gyro_x = %4.3fdps, gyro_y = %4.3fdps, gyro_z = %4.3fdps\r\n", gyro[0],
50 gyro[1], gyro[2]);
51     printf("tim_count = %d\r\n", tim_count);
52     Paint_DrawString_EN(5, 10, "6DOF TEST", &Font12, WHITE, BLACK);
53     Paint_DrawString_EN(5, 25, "IC:QMI8658", &Font12, WHITE, BLACK);
54
55     sprintf(str, "acc_x :%.2f", acc[0]);
56     Paint_DrawString_EN(5, 40, str, &Font12, WHITE, BLACK);
57     sprintf(str, "acc_y :%.2f", acc[1]);
58     Paint_DrawString_EN(5, 55, str, &Font12, WHITE, BLACK);
59     sprintf(str, "acc_z :%.2f", acc[2]);
60     Paint_DrawString_EN(5, 70, str, &Font12, WHITE, BLACK);
61
62     sprintf(str, "gyro_x:%.2f", gyro[0]);
63     Paint_DrawString_EN(5, 85, str, &Font12, WHITE, BLACK);
64     sprintf(str, "gyro_y:%.2f", gyro[1]);
65     Paint_DrawString_EN(5, 100, str, &Font12, WHITE, BLACK);
66     sprintf(str, "gyro_z:%.2f", gyro[2]);
67     Paint_DrawString_EN(5, 115, str, &Font12, WHITE, BLACK);
68
69     OLED_1in5_Display(BlackImage);
70
71     DEV_Delay_ms(100);
72     Paint_Clear(BLACK);
73 }
74 }

```

7. Створіть м'ютекс для доступу до спільних апаратних ресурсів (I²C шини).
8. Створіть черги для передачі даних від задач сенсорів до задачі відображення.
9. Створіть задачу, яка буде опитувати кнопку і записувати стан кнопки в чергу. Використайте `gpio_get()` для зчитування стану кнопки. Реалізуйте `debounce` (придушення брязкоту контактів). Передавайте значення `true/false` у чергу `xQueueButton`. Опитуйте кнопку кожні 50 мс.
10. Створіть окремі задачі для зчитування даних із сенсорів: температури та вологості (SHTC3), якості повітря (SGP40), акселерометра та гіроскопа (QMI8658). Кожна задача має: ініціалізувати свій сенсор, зчитувати дані з певним періодом (100мс), передавати дані у чергу.
11. Реалізуйте задачу виведення даних на OLED-дисплей, яка приймає дані з черг і відображає їх у зручному форматі. Оновлюйте екран з певною періодичністю (200–300 мс).
12. Додайте функцію вмикання/вимикання OLED-дисплея через кнопку. Якщо кнопка натиснута – перемикає стан змінної `display_on`. Якщо `display_on == false`, дисплей вимикається (очищується або переходить у `sleep`-режим).